TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN

TRIỂN KHAI VÀ QUẢN TRỊ HỆ THỐNG PHẦN MỀM

ĐỀ 30

HỆ THỐNG QUẢN LÝ ĐỘI XE

LAM FLEET

Sinh viên thực hiện Nguyễn Thị Lam

Mã số sinh viên DTC245200411

Lớp CNTT K23D

Giảng viên hướng dẫn Vũ Việt Dũng

Thái Nguyên tháng 10 năm 2026

MỤC LỤC

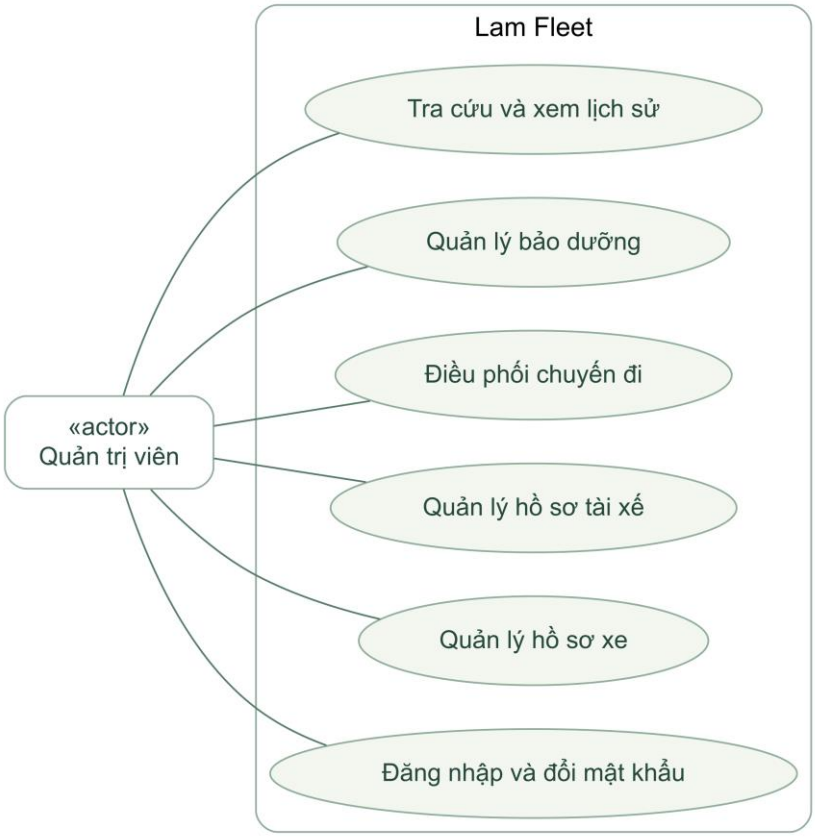
Chương 1 Tổng quan kiến trúc và môi trường **3**
1.1 Mục tiêu và phạm vi 3
1.2 Kiến trúc triển khai WSL2 4
1.3 Quản lý mã nguồn trên GitHub 5
Chương 2 Ứng dụng Database và Reverse Proxy **6**
2.1 Mô hình dữ liệu và ràng buộc 6
2.2 PostgreSQL và pgAdmin 7
2.3 Hồ sơ xe và tài xế 8
2.4 Điều phối và chống trùng lịch 9
2.5 Vòng đời bảo dưỡng 10
2.6 Nginx và cách khởi động 11
Chương 3 Giám sát và log tập trung **12**
3.1 Prometheus và exporter 12
3.2 Dashboard Grafana 13
3.3 Loki Promtail và LogQL 14
Chương 4 Hardening và đánh giá **15**
4.1 Biện pháp gia cố 15
4.2 Kết quả kiểm thử 16
4.3 Sao lưu và phục hồi 17
Kết luận kịch bản demo và tài liệu tham khảo 18

Báo cáo trình bày hệ thống đã triển khai bằng Docker Engine trong Ubuntu WSL2. Các ảnh, số liệu và kết quả kiểm tra là minh chứng của project de30lam; hồ sơ xe và tài xế trong dữ liệu khởi tạo là nội dung minh họa.

CHƯƠNG 1 TỔNG QUAN KIẾN TRÚC VÀ MÔI TRƯỜNG

1.1 Mục tiêu và phạm vi

Lam Fleet phục vụ một quản trị viên quản lý phương tiện, tài xế, chuyến đi và bảo dưỡng. Người quản trị xem tình trạng đội xe, xếp lịch theo khoảng thời gian và lưu kết quả công việc. Mục tiêu vận hành là triển khai đồng bộ ứng dụng, database, công cụ quản trị, reverse proxy, giám sát và log tập trung bằng Docker Compose.



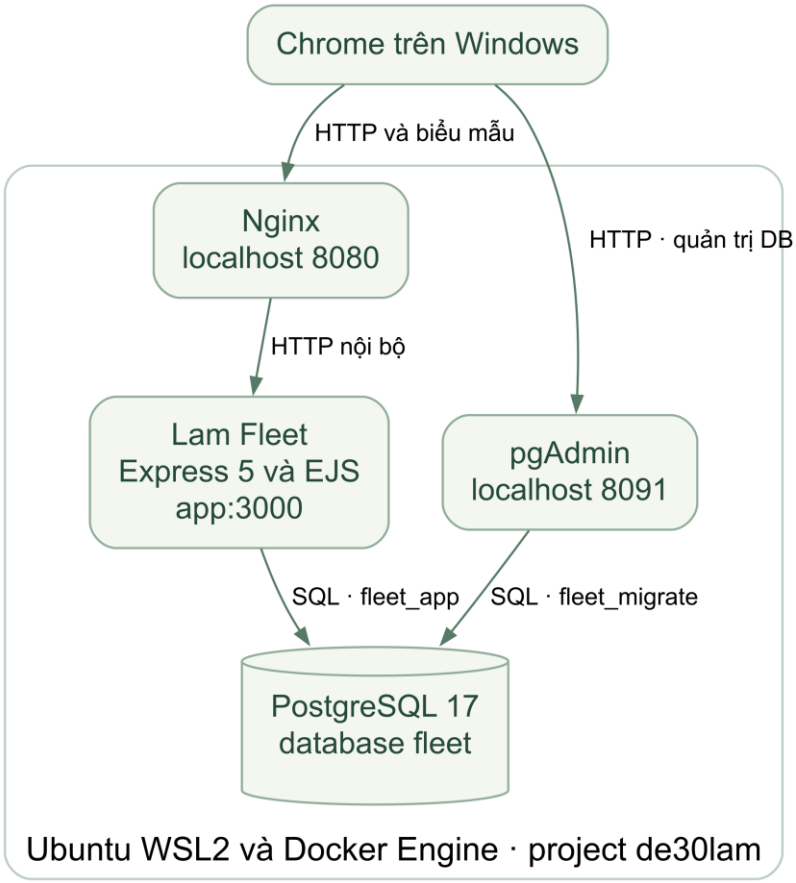
Hình 1 Các nhóm chức năng của quản trị viên

Phạm vi bài thực hành gồm một đơn vị quản lý đội xe, một tài khoản quản trị chính và dữ liệu mẫu. Hệ thống không tích hợp GPS, thanh toán, định tuyến bản đồ hoặc ứng dụng di động. Giới hạn này giúp tập trung kiểm chứng khả năng điều phối và quản trị hạ tầng.

Tiêu chí hoàn thành là website thao tác được qua Nginx ở cổng 8080, pgAdmin kết nối database fleet, Grafana hiển thị metrics thực, truy vấn LogQL có dữ liệu và các biện pháp hardening có kết quả kiểm tra. Báo cáo đi kèm mã nguồn và hướng dẫn tự chạy lại.

1.2 Kiến trúc triển khai WSL2

Chrome chạy trên Windows, còn toàn bộ dịch vụ nằm trong Docker Engine của Ubuntu WSL2. Compose project de30lam có network và volume riêng; việc triển khai không thay đổi các bài thực hành khác đang chạy. Nguồn chính đặt trong filesystem Linux để giữ quyền truy cập và hiệu năng mount.



Hình 2 Kiến trúc triển khai ứng dụng và database

Dịch vụ	Địa chỉ trên Windows	Cách truy cập
Website	localhost:8080	Qua Nginx
pgAdmin	localhost:8091	admin@lam.edu.vn
Grafana	localhost:3005	admin
Prometheus	localhost:9094	Trang Targets và truy vấn

PostgreSQL và app không publish cổng ra host. Các cổng giao diện chỉ bind 127.0.0.1. Khi demo từ Windows, cần giữ phiên Ubuntu hoạt động để dịch vụ và chuyển tiếp localhost không bị gián đoạn.

1.3 Quản lý mã nguồn trên GitHub

Repository riêng tư thuộc tài khoản GitHub DTC245200411, trùng mã số sinh viên. Trước khi tạo repository, GitHub MCP đã trả đúng tài khoản này; Git Credential Manager trên Windows và kết nối Git trong WSL cũng được kiểm chứng qua GitHub API. Git dùng tác giả Nguyễn Thị Lam và email noreply của đúng tài khoản.

Repository bàn giao có địa chỉ https://github.com/DTC245200411/dtc245200411-de30-quan-ly-doi-xe. Nhánh bàn giao trên GitHub là main. Mỗi mốc có commit riêng phản ánh đúng phần việc được đưa vào mã nguồn, thay vì tạo lịch sử rỗng để đủ số lượng.

Mốc	Nội dung thực hiện	Minh chứng
1	Ứng dụng Fleet, PostgreSQL, pgAdmin, Nginx, xác thực và kiểm thử nghiệp vụ	Commit 70b2173
2	Prometheus, Grafana, exporter và số liệu container thuộc de30lam	Commit 8d2fa4a
3	Loki, Promtail, LogQL, hardening, backup và tài liệu bàn giao	Commit hoàn thiện trên main

Thư mục app chứa schema, migration, domain, giao diện và kiểm thử. infra chứa cấu hình PostgreSQL, pgAdmin, Nginx và xoay vòng log. observability chứa các datasource, dashboard và cấu hình thu thập. scripts cung cấp thao tác vận hành; docs giữ sơ đồ và minh chứng.

Các tệp .env, .local, secret và bản backup không được commit. .env.example chỉ khai báo cổng mẫu, không có mật khẩu. Script chuẩn bị sinh mật khẩu riêng khi chạy lần đầu và giữ lại khi chạy tiếp. Dependency được khóa bằng package-lock.json; image Docker dùng tag phiên bản cụ thể.

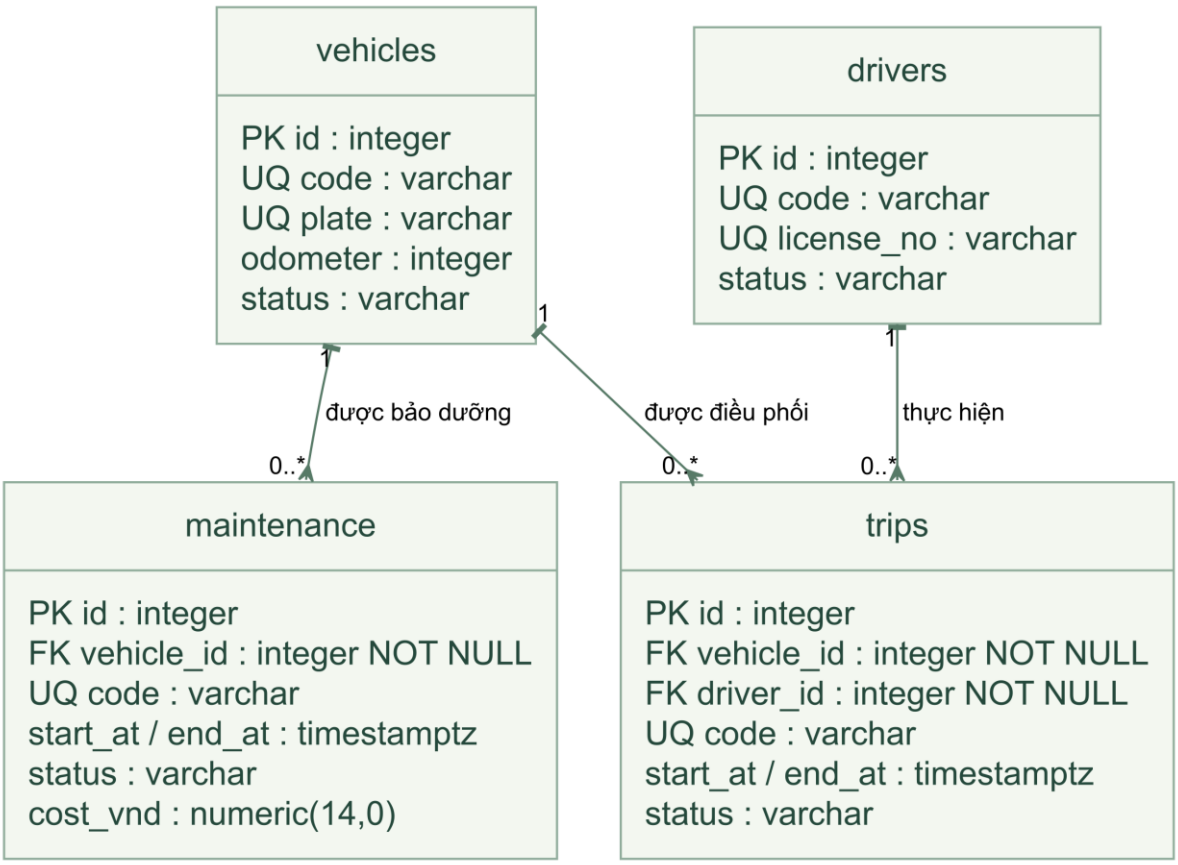
```
git log --oneline
git status --short
git remote -v
```

Người clone repository riêng tư phải có quyền truy cập. README là tài liệu vận hành đầu tiên; sơ đồ có nguồn DOT để chỉnh sửa và tái tạo.

CHƯƠNG 2 ỨNG DỤNG DATABASE VÀ REVERSE PROXY

2.1 Mô hình dữ liệu và ràng buộc

PostgreSQL lưu bốn nhóm nghiệp vụ chính. Mỗi chuyến bắt buộc tham chiếu một xe và một tài xế; mỗi phiếu bảo dưỡng tham chiếu một xe. Một xe hoặc tài xế có thể chưa có lịch sử, vì vậy phía tập lịch có lực lượng 0..n.



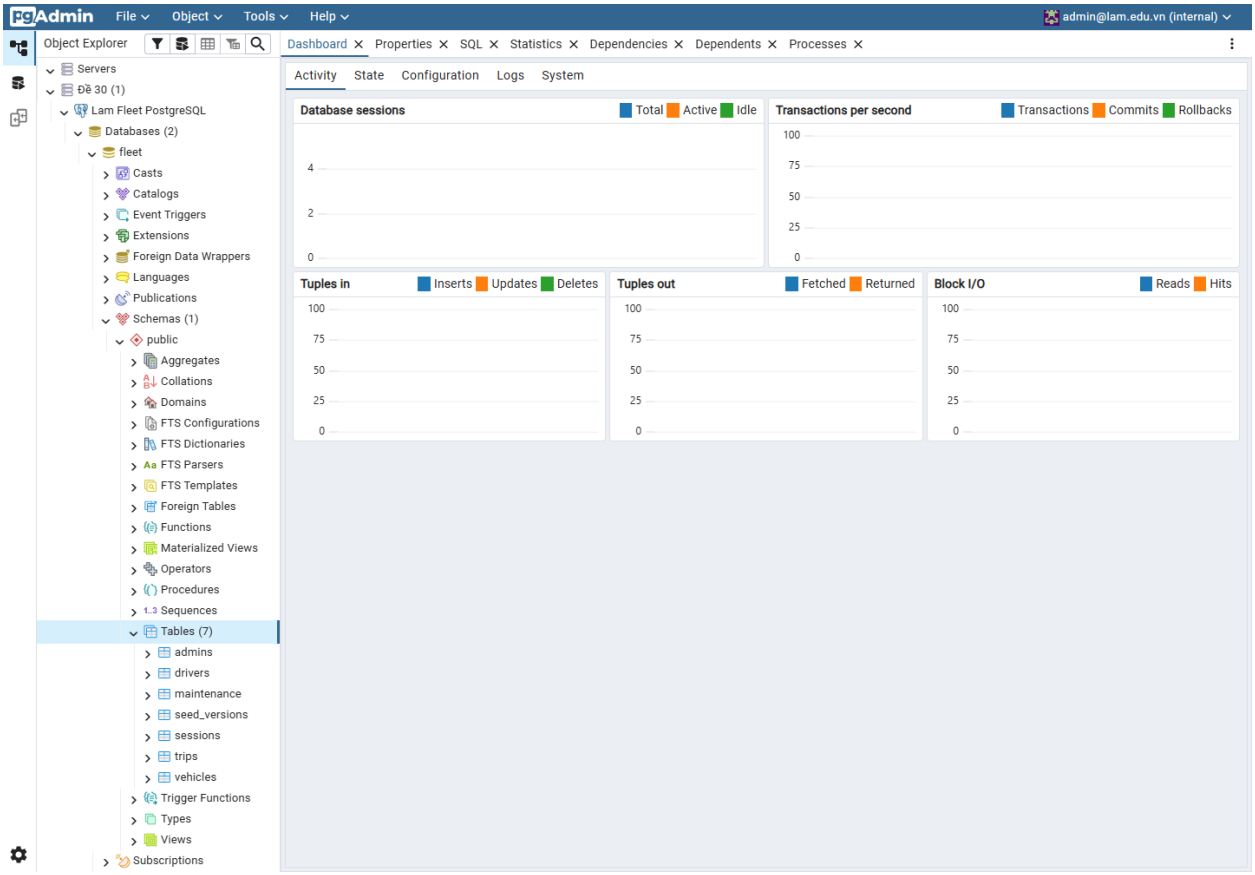
Hình 3 ERD vật lý rút gọn của bốn bảng nghiệp vụ

Bảng bổ trợ	Vai trò	Ràng buộc chính
admins	Tài khoản quản trị	username duy nhất, mật khẩu băm scrypt
sessions	Phiên đăng nhập	sid là PK; sess là JSON; không có FK đến admins
seed_versions	Đánh dấu seed đã áp dụng	version là PK; seed không chạy lặp

Các mã xe, tài xế, chuyến và phiếu đều duy nhất; biển số và số giấy phép cũng duy nhất. start_at và end_at dùng timestamptz, với end_at lớn hơn start_at. Số kilomet và chi phí không âm. Chỉ mục hỗ trợ tra lịch mở theo xe, tài xế và thời gian; version hỗ trợ phát hiện cập nhật từ biểu mẫu cũ.

2.2 PostgreSQL và pgAdmin

Database fleet chạy trên PostgreSQL 17. Role fleet_owner phục vụ khởi tạo và sao lưu; fleet_migrate tạo schema và seed; fleet_app thực hiện nghiệp vụ theo quyền được cấp; fleet_exporter chỉ phục vụ giám sát. Ứng dụng không có quyền tạo bảng hoặc xóa lịch sử chuyến và bảo dưỡng.



Hình 4 pgAdmin kết nối thành công tới database fleet

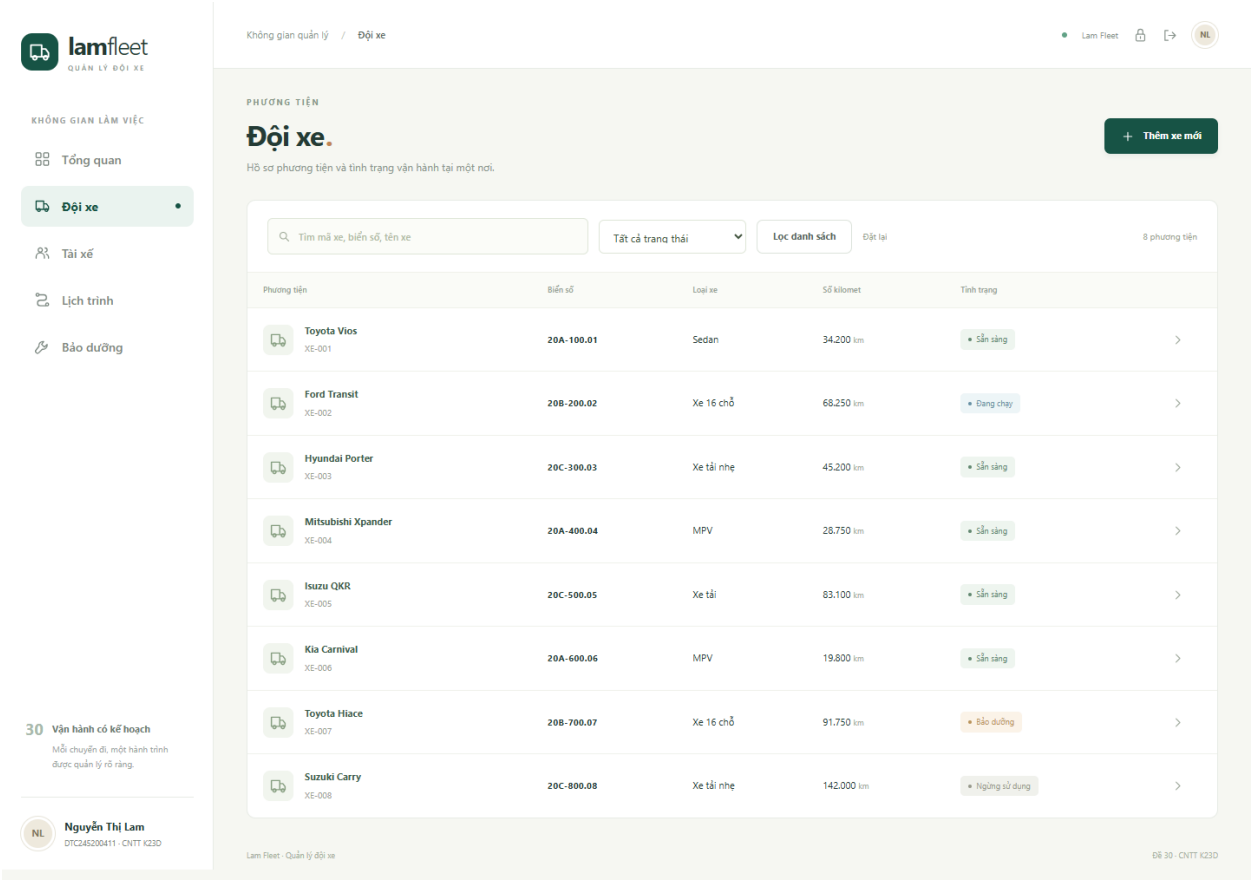
pgAdmin truy cập tại cổng 8091 với email admin@lam.edu.vn. Server Lam Fleet PostgreSQL được khai báo sẵn, trỏ postgres:5432, database fleet, tài khoản fleet_migrate. Mật khẩu đăng nhập pgAdmin và mật khẩu kết nối database là hai secret khác nhau; cả hai chỉ lưu trong .local/secrets.

Migration và seed chạy trong transaction, có advisory lock để không khởi tạo đồng thời. Phiên bản seed chỉ được đánh dấu khi toàn bộ dữ liệu ghi thành công. Bộ dữ liệu gồm 8 xe, 6 tài xế, 10 chuyến và 5 phiếu bảo dưỡng; có xe ngừng sử dụng và nhiều trạng thái lịch để demo.

```
SELECT status, count(*) FROM trips GROUP BY status;
SELECT status, count(*) FROM maintenance GROUP BY status;
```

2.3 Hồ sơ xe và tài xế

Trang Đội xe tổng hợp mã xe, tên, biển số, loại, số kilomet và tình trạng vận hành. Trạng thái sẵn sàng, đang chạy hoặc đang bảo dưỡng được suy ra từ hồ sơ và công việc thực tế đang mở, giúp tránh lưu hai nguồn trạng thái mâu thuẫn. Người dùng tìm theo mã, biển số hoặc tên xe.



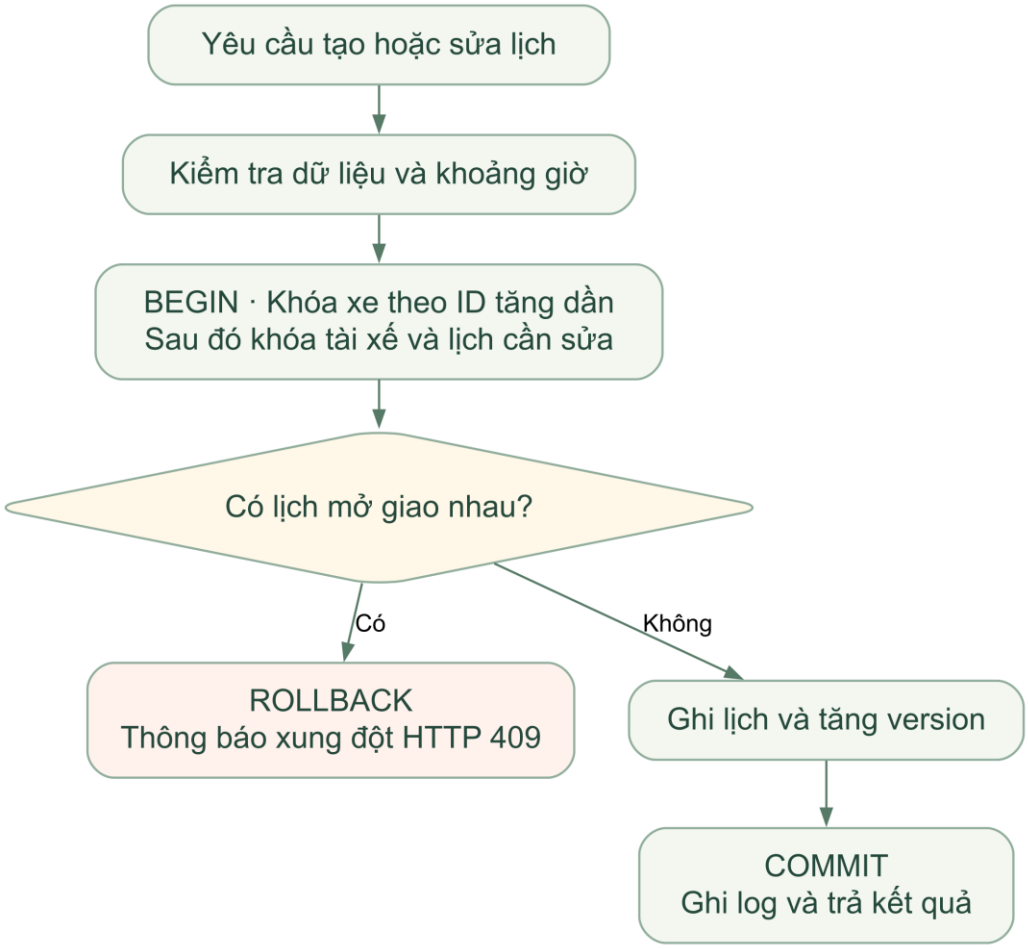
Hình 5 Danh sách phương tiện và trạng thái vận hành

Hồ sơ tài xế có mã duy nhất, họ tên, điện thoại và số giấy phép. Người quản trị được sửa thông tin và chuyển ngừng hoạt động khi không còn lịch mở. Trang chi tiết liên kết tới các chuyến đã thực hiện; số điện thoại và giấy phép trong seed chỉ là dữ liệu minh họa.

Khi cập nhật xe, hệ thống không chấp nhận kilomet giảm. Nếu xe đang thực hiện công việc, kilomet chỉ được thay đổi khi hoàn tất công việc đó. Hồ sơ được khóa trong transaction, đồng thời kiểm tra version để một biểu mẫu cũ không ghi đè thay đổi mới. Hệ thống giữ lịch sử và không cung cấp thao tác xóa vật lý hồ sơ nghiệp vụ.

2.4 Điều phối và chống trùng lịch

Một lịch được biểu diễn bằng khoảng nửa mở [bắt đầu, kết thúc). Hai lịch tiếp giáp được chấp nhận; hai lịch có giao nhau trên cùng xe hoặc cùng tài xế bị từ chối. Kiểm tra xe bao gồm cả chuyến đi và bảo dưỡng ở trạng thái dự kiến hoặc đang thực hiện.



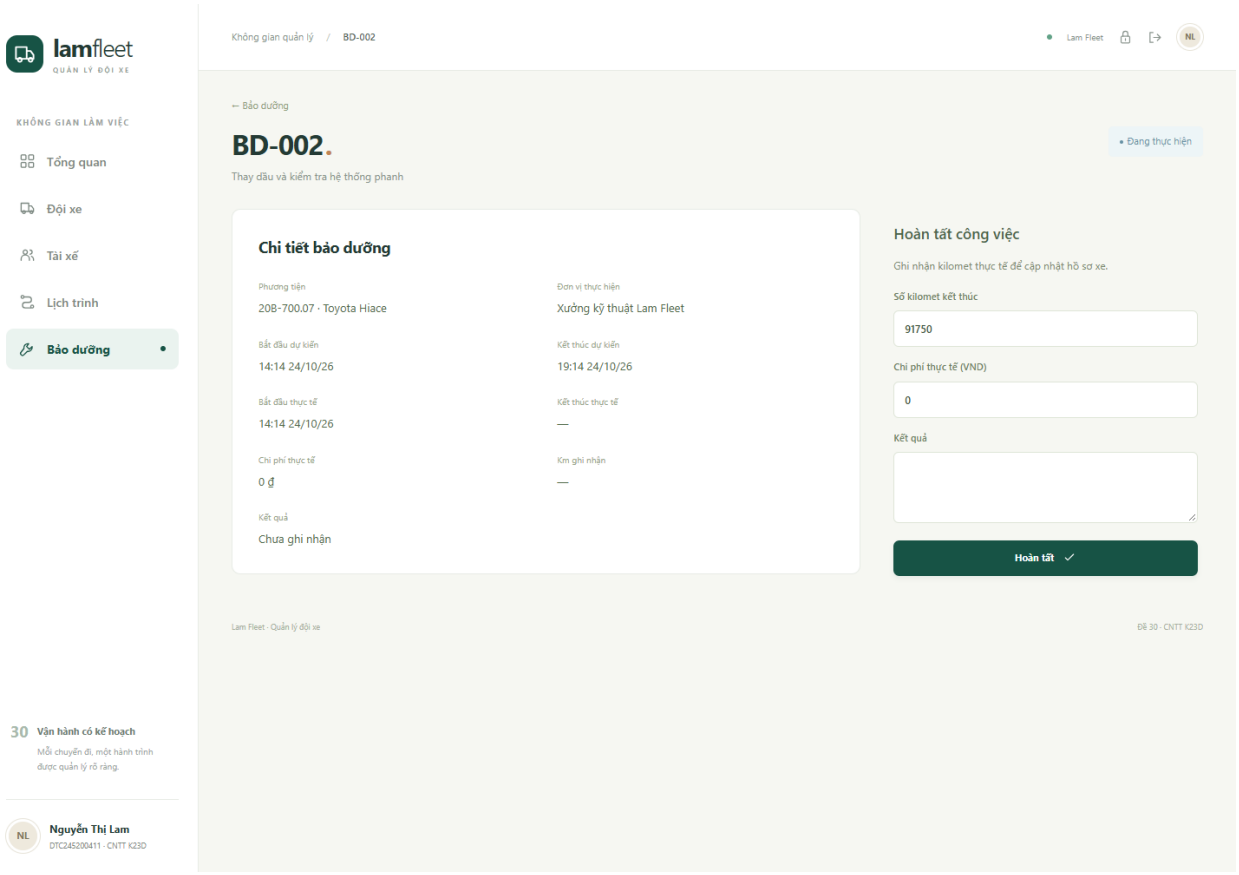
Hình 6 Transaction và thứ tự khóa khi ghi lịch

Các thao tác khóa xe theo ID tăng dần trước, khóa tài xế sau và cuối cùng khóa lịch khi sửa. Sau khi lấy khóa, hệ thống đọc lại dữ liệu và kiểm tra giao nhau trước khi ghi. Hai yêu cầu cùng đặt một tài nguyên vì thế có tối đa một yêu cầu thành công; yêu cầu còn lại nhận HTTP 409 cùng thông báo tiếng Việt.

Chuyến dự kiến có thể sửa hoặc hủy. Khi bắt đầu, hệ thống kiểm tra thêm công việc đang chạy, kể cả trường hợp công việc đó đã quá giờ dự kiến. Khi hoàn tất, kilomet cuối không được giảm và được cập nhật về hồ sơ xe. Bài không tự động điều chỉnh lịch khác khi một chuyến thực tế kéo dài.

2.5 Vòng đời bảo dưỡng

Phiếu bảo dưỡng lưu mã phiếu, xe, nội dung, đơn vị thực hiện và khoảng thời gian dự kiến. Phiếu chưa bắt đầu có thể chỉnh sửa hoặc hủy. Khi bắt đầu, xe không được đồng thời có chuyến hoặc bảo dưỡng khác đang thực hiện. Trang tổng quan đưa các lịch mở lên để người quản trị xử lý, đánh dấu lịch dự kiến đã quá giờ bắt đầu.



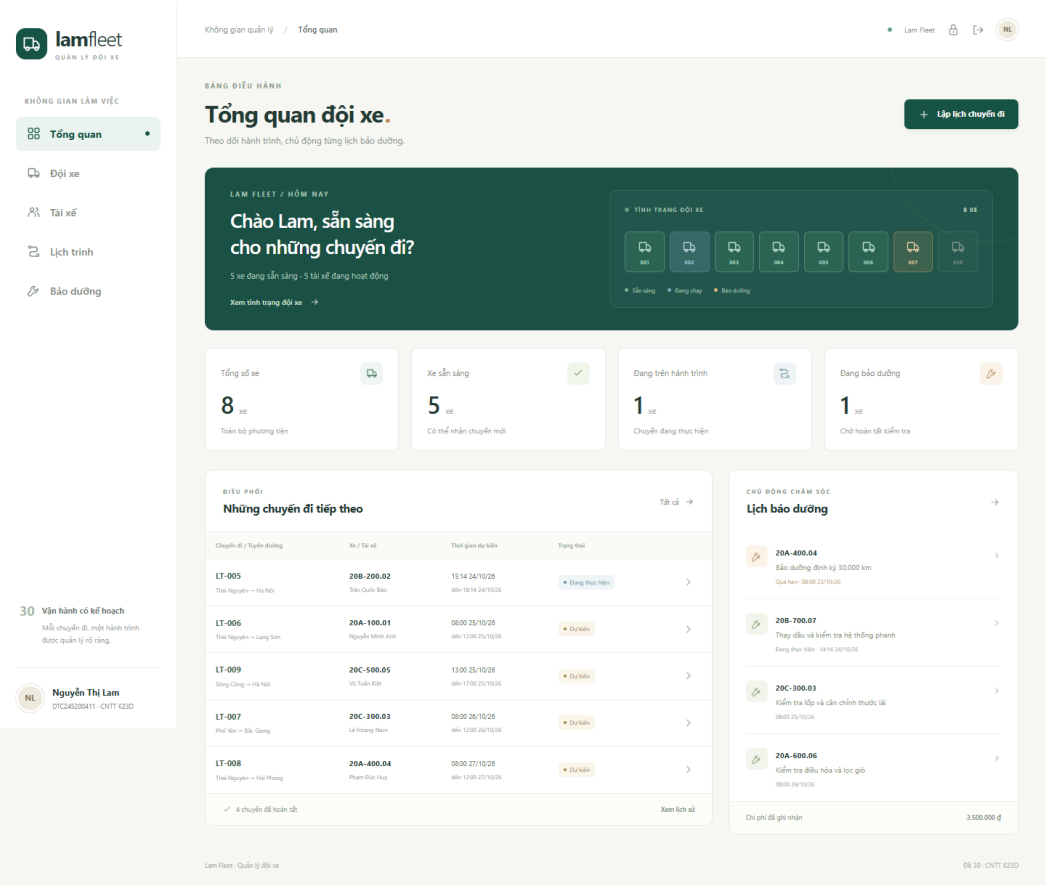
Hình 7 Ghi nhận hoàn tất bảo dưỡng và chi phí thực tế

Hoàn tất bảo dưỡng cần số kilomet hợp lệ, chi phí VND nguyên không âm và kết quả thực hiện. Thay đổi trạng thái và cập nhật kilomet xe nằm trong cùng transaction. Phiếu đã hoàn tất giữ nguyên lịch sử để tra cứu; tổng chi phí trên dashboard chỉ cộng các phiếu đã hoàn tất.

Cảnh báo quá hạn dựa trên giờ bắt đầu dự kiến, không phải cảnh báo định kỳ theo kilomet. Việc lập kế hoạch lần bảo dưỡng kế tiếp do quản trị viên thực hiện. Đây là phạm vi đủ để minh họa nghiệp vụ quản lý và các sự kiện log trong bài thực hành.

2.6 Nginx và cách khởi động

Nginx là cổng truy cập website duy nhất ở localhost:8080 và chuyển tiếp tới app:3000 trong mạng frontend. Cấu hình có giới hạn kích thước biểu mẫu, timeout và các security header. Các endpoint /metrics và /readyz bị chặn ở cổng website; exporter truy cập qua mạng monitoring riêng.



Hình 8 Trang tổng quan Lam Fleet được phục vụ qua Nginx

```
sh scripts/start.sh
sh scripts/test.sh
sh scripts/stop.sh
```

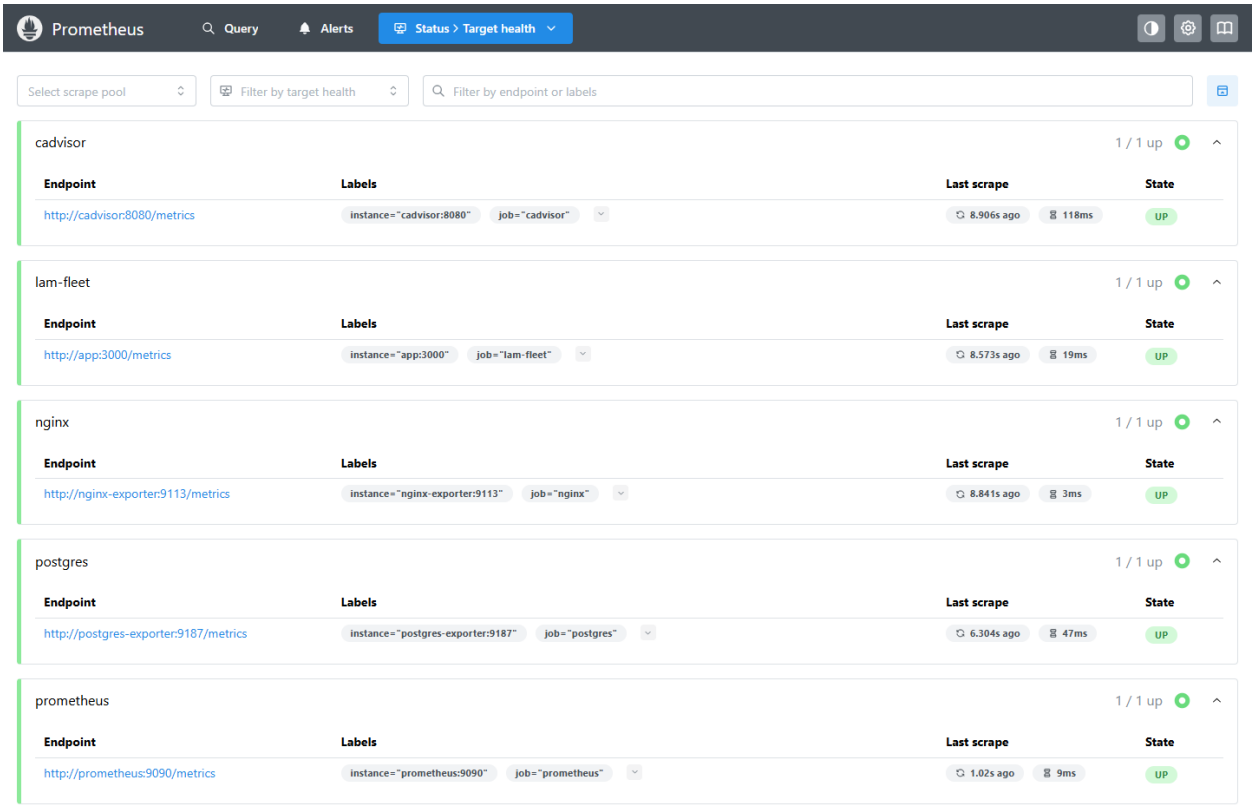
Script start chuẩn bị secret, kiểm tra Compose, build image, đợi dịch vụ sẵn sàng và kiểm tra cấu hình Nginx. Có thể dùng BASE_ONLY=1 để chỉ chạy ứng dụng và database. PowerShell cung cấp wrapper gọi đúng Ubuntu WSL2. Khi dùng trực tiếp lệnh Linux, giữ cửa sổ Ubuntu mở trong suốt buổi demo.

HTTP localhost cùng security headers đáp ứng lựa chọn triển khai của đề. Khi đưa ra Internet cần bổ sung chứng chỉ HTTPS hợp lệ, cookie Secure và kiểm soát các cổng quản trị; cấu hình demo hiện tại không được mô tả là đã triển khai HTTPS.

CHƯƠNG 3 GIÁM SÁT VÀ LOG TẬP TRUNG

3.1 Prometheus và exporter

Prometheus thu thập số liệu mỗi 15 giây từ năm job: prometheus, lam-fleet, nginx, postgres và cadvisor. App công bố metrics HTTP và trạng thái nghiệp vụ. Exporter Nginx đọc stub_status nội bộ; exporter PostgreSQL dùng role giám sát. cAdvisor cung cấp số liệu container trên Docker host.



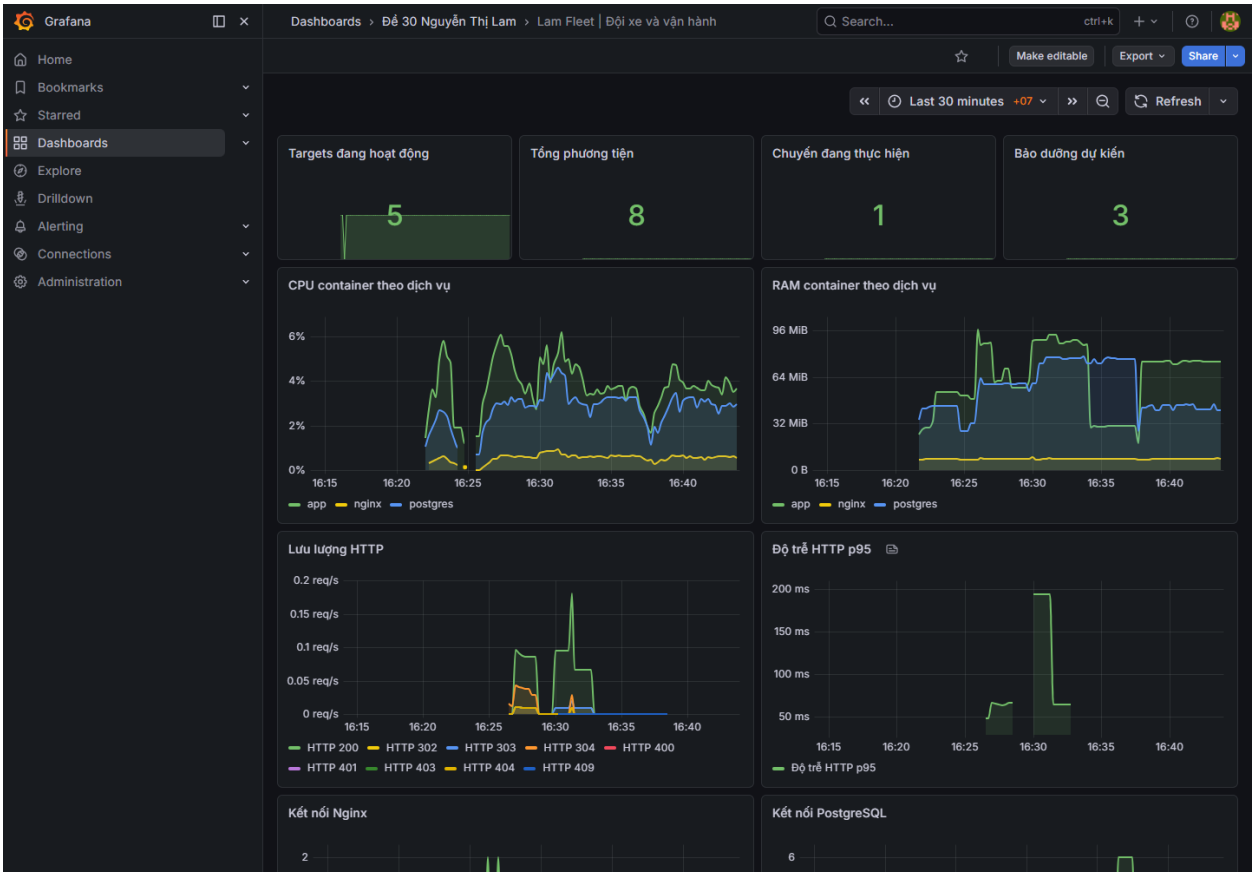
Hình 9 Các target của project Lam Fleet trong Prometheus

Kết quả kiểm tra xác nhận đủ 5 target UP, nginx_up và pg_up bằng 1. CPU và RAM phải có nhãn Compose project de30lam và đúng service app, nginx, postgres; số liệu chỉ thuộc Docker host hoặc bài khác không được coi là đạt. Gauge tổng xe đọc trực tiếp PostgreSQL và bằng 8.

Biểu đồ rate cần ít nhất hai lần thu thập. Các bộ đếm HTTP và thao tác bắt đầu lại khi app restart; số chuyến và phiếu trong database vẫn giữ. Dữ liệu Prometheus được giữ 7 ngày và giới hạn dung lượng theo cấu hình demo.

3.2 Dashboard Grafana

Grafana được provision tự động với datasource Prometheus và Loki. Dashboard Lam Fleet Đội xe và vận hành nằm trong thư mục Đề 30 Nguyễn Thị Lam, gồm 14 panel metrics và 3 panel log. Người dùng không phải tạo thủ công dashboard sau khi clone dự án.



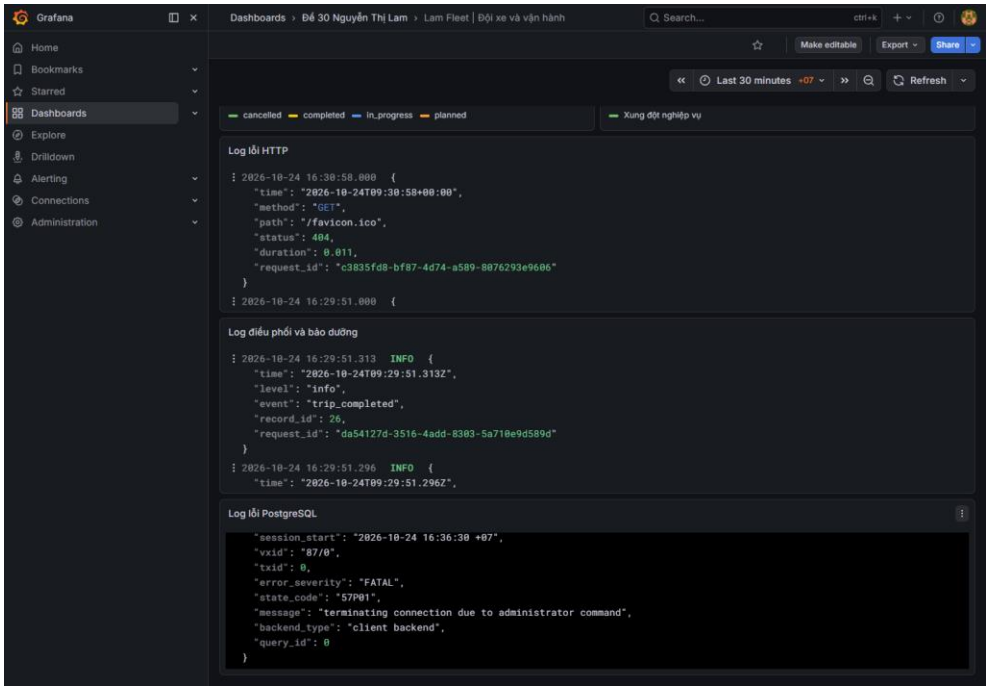
Hình 10 Dashboard với số liệu CPU RAM HTTP và database

Các panel đầu hiển thị số target đang hoạt động, tổng xe, chuyến đang thực hiện và bảo dưỡng dự kiến. Nhóm biểu đồ theo dõi CPU/RAM, lưu lượng HTTP theo mã phản hồi, p95 độ trễ, kết nối Nginx, kết nối và giao dịch PostgreSQL. Nhóm nghiệp vụ thể hiện trạng thái chuyến, bảo dưỡng và số yêu cầu xung đột bị từ chối.

Truy vấn container luôn lọc project de30lam. Dashboard phục vụ việc đối chiếu tác động của thao tác demo lên hệ thống, không thay thế sổ lịch trình trong database. Alert target mất kết nối được khai báo trong Prometheus; bài không cấu hình gửi email hay thông báo ra dịch vụ bên ngoài.

3.3 Loki Promtail và LogQL

Ứng dụng, Nginx và PostgreSQL ghi log ra volume riêng. Promtail mount log ở chế độ chỉ đọc, lưu vị trí đọc trên volume độc lập rồi đẩy vào Loki. Grafana dùng LogQL để tìm sự kiện. Loki giữ log 7 ngày; logrotate quản lý file nhằm hạn chế tăng dung lượng trên máy demo.



Hình 11 Truy vấn log nghiệp vụ và lỗi database trên Grafana

Lỗi HTTP

```
{job="nginx", project="de30lam"}
  | json | status >= 400
```

Thao tác nghiệp vụ

```
{job="app", project="de30lam"} | json
  | event=~"(trip|maintenance)_(scheduled|started|completed|cancelled)"
```

Lỗi PostgreSQL

```
{job="postgres", project="de30lam"} | json
  | error_severity=~"ERROR|FATAL|PANIC"
```

Lần kiểm chứng thu được lần lượt 20, 20, 18 sự kiện theo giới hạn truy vấn. Mật khẩu, cookie và nội dung form không được đưa vào log. Promtail được giữ theo yêu cầu bài; Grafana xác nhận công cụ đã hết hỗ trợ ngày 02/03/2026, nên hướng nâng cấp là Alloy [4].

CHƯƠNG 4 HARDENING VÀ ĐÁNH GIÁ

4.1 Biện pháp gia cố

Gia cố được áp dụng ở cả ứng dụng, container và database. Script check-hardening.py đọc cấu hình container đang chạy để xác nhận quyền người dùng, filesystem, capability, network và cách publish cổng, thay vì chỉ dựa trên khai báo trong báo cáo.

Lớp	Biện pháp	Kết quả đối chiếu
Ứng dụng	Mật khẩu băm scrypt, CSRF, giới hạn đăng nhập, kiểm tra dữ liệu	HTTP test đạt
Phiên	Cookie HttpOnly và SameSite; đổi mật khẩu làm hết hiệu lực phiên cũ	HTTP test đạt
App và Nginx	Non-root, read-only rootfs, cap_drop ALL, no-new-privileges	Container inspect đạt
PostgreSQL	Tách role; app không có CREATE schema hoặc DELETE lịch sử	Kiểm tra quyền đạt
Network	Database nội bộ; không publish app, DB, Loki hoặc exporter	Container inspect đạt
Cổng giao diện	Website và công cụ chỉ bind 127.0.0.1	Port binding đạt
Secret	Sinh riêng, thư mục 0700, không có trong Git	Kiểm tra filesystem và Git đạt
Log	Mount log Promtail chỉ đọc; không mount Docker socket	Container inspect đạt

Các ngoại lệ được ghi rõ: cAdvisor cần privileged và truy cập Docker host; storage-init dùng root tạm thời để đặt quyền volume; PostgreSQL bootstrap bắt đầu bằng root rồi chuyển sang postgres; logrotate có một số capability phục vụ file log. Vì vậy không khẳng định tất cả container đều non-root hoặc không có đặc quyền.

Các biện pháp trên phù hợp môi trường demo nội bộ. Chưa thực hiện kiểm thử xâm nhập toàn diện hay đánh giá tải lớn. Trước khi cung cấp dịch vụ công khai cần bổ sung HTTPS, chính sách truy cập quản trị, cập nhật công cụ log còn hỗ trợ và kế hoạch sao lưu ngoài máy.

4.2 Kết quả kiểm thử

Bộ kiểm thử dùng Node test runner, gửi HTTP thật qua Nginx và đối chiếu PostgreSQL. Mỗi lần chạy tạo tài khoản QA và dữ liệu có tiền tố riêng, sau đó dọn dữ liệu thử. Mật khẩu của tài khoản admin chính không bị thay đổi. Kết quả lần bàn giao là 29/29 trường hợp đạt, không có trường hợp bỏ qua.

Nhóm kiểm thử	Số ca	Kết quả
Xác thực CSRF headers cookie và đổi mật khẩu	5	Đạt
Màn hình và tìm kiếm tham số hóa	2	Đạt
Dữ liệu trạng thái quyền DB và seed	6	Đạt
Tạo sửa hủy và hoàn tất chuyến	5	Đạt
Bảo dưỡng chi phí và công việc đang chạy	3	Đạt
Yêu cầu đồng thời	4	Đạt
Kiểm tra số mã và thời gian	4	Đạt
Tổng cộng	29	29 đạt 0 lỗi

Các ca đồng thời gửi hai yêu cầu cùng đặt xe, cùng đặt tài xế, một chuyến cạnh tranh với một phiếu bảo dưỡng và hai lần bắt đầu cùng một lịch. Mỗi trường hợp có đúng một yêu cầu thành công; yêu cầu còn lại bị từ chối, không ghi dữ liệu một phần. Lịch tiếp giáp vẫn được chấp nhận.

```
# tests 29
# pass 29
# fail 0
# skipped 0
```

Kiểm thử trình duyệt Chrome chụp các màn hình và xác nhận không có lỗi JavaScript ở website. Màn hình desktop và viewport 390 px không gây tràn ngang toàn trang; bảng dài có vùng cuộn riêng. npm audit trên dependency đã khóa báo 0 lỗ hổng tại thời điểm triển khai. Những kết quả này kiểm chứng phạm vi đã thử, không phải cam kết không còn mọi lỗi bảo mật.

4.3 Sao lưu và phục hồi

Script backup dùng pg_dump dạng custom để lưu schema, dữ liệu, ownership và ACL. Mỗi archive đi kèm SHA-256; thư mục backup không nằm trong Git và được tạo với quyền hạn chế. File có dữ liệu tài khoản và phiên nên không được chia sẻ như minh chứng công khai.

```
sh scripts/backup.sh
sh scripts/restore.sh --verify-current .local/backups/<tep>.dump
```

Chế độ verify-current kiểm tra checksum, tạo database tạm, phục hồi archive không ghi đè database đang dùng, rồi so sánh fingerprint của bốn bảng nghiệp vụ. Database tạm được dọn sau kiểm tra. So sánh này xác nhận nội dung dữ liệu nghiệp vụ khớp, không chỉ đếm số dòng.

Bảng nghiệp vụ	Số dòng phục hồi	Đối chiếu
vehicles	8	Khớp
drivers	6	Khớp
trips	10	Khớp
maintenance	5	Khớp

Fingerprint đã đối chiếu: ac7fdcd5ea89614274b887cccb7319ba. Kết quả ghi nhận PASS restored business rows match current database exactly trong minh chứng restore.txt.

Chế độ --apply phục hồi thay dữ liệu hiện có. Script kiểm tra archive bằng database tạm trước, yêu cầu nhập PHUC HOI LAM FLEET, dừng app/Nginx rồi mới phục hồi. Nếu thất bại, ứng dụng giữ trạng thái dừng để kiểm tra; không tiếp tục phục vụ với dữ liệu chưa rõ trạng thái. Bài bàn giao đã thử phục hồi vào database tạm, không thực hiện ghi đè database demo.

Dừng hệ thống thông thường giữ named volume. Khởi động lại chạy migration và kiểm tra seed_versions, nên không nhân đôi dữ liệu hoặc đặt lại mật khẩu. Không sử dụng lệnh prune toàn máy hoặc xóa volume của bài khác để xử lý lỗi.

KẾT LUẬN KỊCH BẢN DEMO VÀ TÀI LIỆU THAM KHẢO

Lam Fleet đã tích hợp ứng dụng quản lý đội xe, PostgreSQL, pgAdmin, Nginx, Prometheus, Grafana và Loki/Promtail bằng Docker Compose. Kết quả có 29 kiểm thử đạt, 5 target UP, metrics đúng project, ba truy vấn log có dữ liệu và bản backup phục hồi khớp dữ liệu nghiệp vụ. Mã nguồn và cấu hình được bàn giao trên repository riêng của sinh viên.

Bước	Thao tác demo	Kết quả cần quan sát
1	Mở localhost:8080 và đăng nhập	Dashboard có 8 xe và trạng thái vận hành
2	Tạo xe/tài xế thử và lập chuyến	Lịch lưu thành công và xuất hiện trong danh sách
3	Lập lịch chồng xe hoặc tài xế	Thông báo xung đột, dữ liệu cũ giữ nguyên
4	Bắt đầu rồi hoàn tất chuyến/bảo dưỡng	Trạng thái và kilomet cập nhật đồng bộ
5	Mở pgAdmin Prometheus và Grafana	Database fleet, 5 target UP, metrics và log
6	Chạy kiểm thử và verify backup	Kết quả PASS và dữ liệu phục hồi khớp

Hướng phát triển gồm phân quyền nhiều người dùng, nhắc bảo dưỡng theo kilomet, xử lý chuyến kéo dài ảnh hưởng lịch sau, HTTPS và chuyển Promtail sang Grafana Alloy. Bản hiện tại không có GPS, thanh toán hoặc kiểm chứng tải lớn. Khi cần chạy lại, bắt đầu từ README và kịch bản demo trong thư mục docs.

Tài liệu tham khảo

[1] Đề tài thực hành môn Triển khai và Quản trị Hệ thống Phần mềm K23, yêu cầu chung và đề 30, tài liệu PDF được cung cấp.

[2] Docker Compose. https://docs.docker.com/compose/

[3] PostgreSQL 17 Explicit Locking. https://www.postgresql.org/docs/17/explicit-locking.html

[4] Grafana Promtail agent. https://grafana.com/docs/loki/latest/send-data/promtail/

[5] Prometheus Overview. https://prometheus.io/docs/introduction/overview/